

BORNFIRE

DATABASE SCHEMA & DATA MODEL GUIDE



**TECHNICAL
REFERENCE**

**AUTHOR: BORNFIRE
ENGINEERING**

**TARGET AUDIENCE: INTERNS / NEW
DEVELOPERS**

1. DATABASE OVERVIEW

Bornfire uses a **PostgreSQL** relational database schema managed through **Supabase**. For local development, a Docker container runs PostgreSQL (exposed on host port 5435). The tables, indices, constraints, and triggers are defined in the schema file located in the repository at `backend/schema.sql`.

The database design separates authentication details from user productivity profiles, and groups core operational entities into tasks, habits, gamification elements, and squad friendship connections.

2. TABLE SCHEMA REFERENCE

2.1 TABLE: USERS

Stores authentication details and user roles. Maintained in sync with Supabase Auth schema.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK DEFAULT uuid_generate_v4()	Unique user identifier.
email	TEXT	UNIQUE, NOT NULL	User login email address.
password_hash	TEXT	NOT NULL	Encrypted user password string.
role	TEXT	DEFAULT 'user'	Checked values: 'user' or 'admin'.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp when the record was last modified.

2.2 TABLE: PROFILES

Extends user information to handle gamification, XP progression, streaks, and focus metrics.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK FK users(id) ON DELETE CASCADE	One-to-one relationship back to the main users table.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
username	TEXT	UNIQUE, NOT NULL	Unique display handle chosen by user.
avatar_url	TEXT	NULLABLE	Link to hosted avatar image files.
xp	INTEGER	DEFAULT 0	Accumulated experience points.
level	INTEGER	DEFAULT 1	Progression level calculated from total XP.
league	TEXT	DEFAULT 'bronze'	Clash of Clans inspired league. checked: bronze, silver, gold, platinum, diamond.
streak	INTEGER	DEFAULT 0	Consecutive check-in days.
longest_streak	INTEGER	DEFAULT 0	Historical maximum streak achieved.
total_focus_time	INTEGER	DEFAULT 0	Accumulated session focus duration in seconds.
completed_tasks_count	INTEGER	DEFAULT 0	Counter of all completed tasks.
last_checkin_date	DATE	NULLABLE	The calendar date of the last user check-in.

2.3 TABLE: TASKS

Stores information about user tasks, Pomodoro timers, and task priority metrics.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	 DEFAULT uuid_generate_v4()	Unique identifier for a task.
user_id	UUID	 users(id) ON DELETE CASCADE	Reference to the task owner.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
title	TEXT	NOT NULL	Task description title.
completed	BOOLEAN	DEFAULT FALSE	Flag indicating task completion state.
time_spent	INTEGER	DEFAULT 0	Time focused on this task (in seconds).
estimated_time	INTEGER	NULLABLE	Planned duration target (in seconds).
remaining_time	INTEGER	NULLABLE	Current state of the countdown focus timer.
is_timer_running	BOOLEAN	DEFAULT FALSE	Flag to signify active Pomodoro session timer.
priority	TEXT	DEFAULT 'medium'	Checked priority: low, medium, high.
xp_reward	INTEGER	DEFAULT 10	XP granted upon task completion.

2.4 TABLE: FRIENDS

Stores bidirectional connections representing squad memberships.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
user_id	UUID	  users(id) ON DELETE CASCADE	First user.
friend_id	UUID	  users(id) ON DELETE CASCADE	Second user (squad buddy).
status	TEXT	DEFAULT 'pending'	Status: pending, accepted, blocked.

3. DATABASE TRIGGERS & PERFORMANCE OPTIMIZATION

To ensure high query speeds during real-time squad tracking and dashboard loading, the schema implements custom triggers and indices.

3.1 AUTOMATIC UPDATED TIMESTAMP

The database defines a trigger procedure `update_updated_at_column()` which fires automatically before updates on the `users` or `profiles` tables to keep timestamps accurate.

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE PROCEDURE update_updated_at_column();
```

3.2 DATABASE INDEXES

Critical tables such as `tasks` contain specific index constraints to optimize query lookup times when fetching daily user statistics:

- `idx_tasks_user_id` ON `tasks(user_id)`: Speeds up dashboard and Pomodoro list fetching queries.
- `idx_habits_user_id` ON `habits(user_id)`: Optimizes loading daily habit statistics.